KEY run sheet ·paper dry run

Cut one practice key from start to finish using numbers and dots only, draw its xpub code, then prove both by restoring the key in a wallet. Two people: one **cutter**, one **checker**. Allow about two hours; copying the xpub grid is the slow part.

> **Practice only. This key is made on paper and typed into a computer. It must never hold money, and everything is destroyed at the end.**

On the table

- Eleven dice (2 orange, 5 white, 4 black) and the dice strip
- Number sheets A to D
- One paper seed plate (seed plate design, page 2) and a dark pen
- This sheet

Kept aside

- **From Part C:** a computer with `lock_standin.py` in LOCK's `reference/` folder
- **From Part E:** one paper xpub grid (xpub plate design, page 2) and a phone
- **From Part F:** word sheets A to D, a paper slip, and a wallet app on the phone

Before you start, test the tools with the two worked examples. Scan the filled xpub grid (xpub plate design, page 3) with the phone. Then run the stand-in with the eleven numbers on the seed plate example (seed plate design, page 3) and the bits 1010010: it must answer row 12 = 1313 and fingerprint a07d432c.

The three rules

- **Dots on top = ● = mark the cell. Blank on top = ○ = leave the cell empty.** (Ordinary dice: even ●, odd ○.)
- **Each die has one place:** its colour gives the group, its dot count gives the position, fewest dots on the left. Nobody chooses where a die goes.
- **Each die is thrown once per row.** No second tries.

Part A ·Rows 1 to 11 (do these five steps eleven times)

- ☐ **1** **Throw** all eleven dice once.
- ☐ **2** **Place** each die in its own slot on the strip, without turning it.
- ☐ **3** **Mark the row.** Cutter: for each die showing dots, fill the matching cell in this row of the plate. Leave the others empty.
- ☐ **4** **Find the number.** Orange dice give the sheet (A to D), white dice the row, black dice the column. Write the four digits in the box at the end of the row.
- ☐ **5** **Check.** Checker: compare the plate row with the dice, cell by cell, then confirm the number on the sheet. Tick the log. Only then clear the dice.

Part B ·Row 12, first seven cells

- ☐ **6** **Throw** the two orange and five white dice only. Place them on the strip.
- ☐ **7** **Mark** the first seven cells of row 12. Checker compares them with the dice.
- ☐ **8** **Leave** the four shaded cells and the number box of row 12 empty.

Log

Row	1	2	3	4	5	6	7	8	9	10	11	12
Dots match dice												
Number written												
Final check (Part D)												

Start: _____ End of B: _____ End of D: _____ End of E: _____ End of F: _____

Part C ·The checksum, from the LOCK stand-in

On the real machine this part is switches and lights. For the dry run a computer does the same sum. It takes the plate's numbers and gives back dots and a number. No words are used.

- [ ] **9** **Type** the eleven numbers from the plate, then the seven cells of row 12 as 1 for a dot and 0 for a blank, with no spaces. In the LOCK folder:
  `python3 reference/lock_standin.py n1 n2 … n11 bbbbbbb`
- [ ] **10** **Check what it read.** It lists the eleven rows back as dots and numbers. Checker compares them with the plate.
- [ ] **11** **Read "Row 12".** Its first seven dots must equal the seven cells already marked. If they differ, stop and find the mistake. Then **mark the last four** in the shaded cells.
- [ ] **12** **Find row 12's number** on the number sheets from its eleven cells, write it in the box, and only then compare it with the number on the screen.

With the real LOCK: set the switches from each plate row in turn and press STORE, eleven times; the lights above the switches repeat each row back. Enter the seven dice bits as the twelfth entry. After about two and a half minutes the lights show row 12: check the first seven, punch the last four. Return every switch to 0.

Part D ·Whole-plate check

- [ ] **13** **Checker alone**, with the plate and the number sheets only. For each of the twelve rows: read the dots, find that sheet, row and column, and compare the number printed there with the number in the box.
- [ ] **14** **Tick** the last line of the log. All twelve must agree before going on. On steel, this is the last moment to find a mistake before sealing.

Part E ·The xpub grid

The stand-in has also printed 41 rows, each in four blocks, A to D, as LOCK's lights would show them.

- [ ] **15** **Copy.** One person reads a block aloud, light by light; the other fills a square for each `#`. Rows 1 to 41, blocks A, B, C, D. The squares already printed black or grey are read out too: they must agree, which shows you are on the right row.
- [ ] **16** **Swap roles** every ten rows, and check the row number each time.
- [ ] **17** **Scan** the finished grid with the phone. The text must equal the "Code text" on the screen: compare the fingerprint in square brackets and the last six characters. Write the fingerprint in the box on the grid.

Part F ·Sweep test

This proves the whole chain: that a plate holding only dots and numbers opens the wallet its xpub grid points to. It is the first time words appear.

- ☐ **18** **Look up the words.** A different person takes the plate and the word sheets, finds all twelve numbers, and writes twelve words on the slip.
- ☐ **19** **Restore** the twelve words in the wallet app as an imported wallet. If asked for a type, choose native SegWit (BIP84).
- ☐ **20** **Compare.** In the wallet's details, find the master fingerprint and the wallet's public key (zpub). Both must equal what the phone read from the grid in step 17.
- ☐ **21** **Result:** ☐ match ☐ no match. If there is no match, check the word lookup first, then the plate rows against the screen.

Part G ·Destroy and record

- ☐ **22** **Delete** the wallet from the app. Close the program and clear the screen.
- ☐ **23** **Destroy** the paper plate, the xpub grid and the word slip.
- ☐ **24** **Record** the date, who took part, the result and the times. Never the numbers or the words.

Notes for the next version

Which step was slowest? Where did a mistake happen, and which check caught it? Was any instruction unclear?